

LUCRAREA NR. 12

PACHETE STANDARD ȘI PREDEFINITE

1. Scopul lucrării

Lucrarea prezintă rolul pachetelor predefinite și standardizate în cadrul proiectării sistemelor hardware în VHDL. Se studiază pachetele STANDARD, TEXTIO și STD_LOGIC_1164, după care sunt trecute în revistă pachetele aritmetice și logice ale principalelor firme producătoare de instrumente integrate de proiectare cu VHDL.

2. Considerații teoretice

2.1 Generalități

Pentru a se evita complicarea inutilă a limbajului VHDL și pentru a i se păstra suplețea, există un anumit număr de pachete de importanță majoră care au fost normalizate. Acestea sunt pachetele *standard*.

Pachetele standard nu pot fi modificate de către proiectant. Două dintre aceste pachete (STANDARD și TEXTIO) sunt definite în manualul de referință al VHDL.

Pachetul STD_LOGIC_1164 nu face parte din mediul standard al VHDL definit în manualul de referință. El constituie o normă IEEE de sine stătătoare; a fost normalizat pentru a facilita portabilitatea fișierelor care conțin cod sursă VHDL sintetizabil. Acesta este primul pachet care urmărește acoperirea lipsurilor existente la nivelul semanticii VHDL pentru sinteză. Acțiunea sa este completată de alte pachete, împreună cu care alcătuiește un mediu normalizat pentru sinteză: NUMERIC_STD și NUMERIC_BIT (care conțin definirea unor funcții aritmetice pe tipul BIT finit) sau pachetele VITAL (prin intermediul cărora devine posibilă luarea în considerare a unor valori temporale precise). Cel mai utilizat pachet numeric este, la ora actuală, pachetul STD_LOGIC_ARITH realizat de firma SYNOPSYS, acest pachet devenind un standard *de facto*, întrucât este propus și vândut de către actualul lider al pieței mondiale în materie de instrumente de dezvoltare bazate pe VHDL.

2.2 Pachetul STANDARD

Pachetul STANDARD reunește declarații de tipuri, de sub-tipuri și de funcții extrem de utile (chiar indispensabile) pentru proiectare. De exemplu, acest pachet conține definiția tipului BIT.

La începutul fiecărei unități de concepție există o clauză **use** implicită (pe care proiectantul nu trebuie neapărat s-o scrie). Așadar, acest pachet este accesibil de oriunde (din orice unitate de proiectare) în VHDL.

Alcătuirea sa este următoarea:

- definiția tipului BOOLEAN;
- definiția tipului BIT;
- definiția tipului CHARACTER;
- definiția tipului SEVERITY_LEVEL (pentru instrucțiunile **assert**);
- definiția tipului INTEGER (capetele intervalului de definiție depind de implementare);
- definiția tipului REAL (capetele intervalului de definiție depind de implementare);
- definiția tipului fizic TIME (timpul de simulare);
- specificația funcției NOW care returnează timpul curent al simulării (de tip TIME);
- definiția sub-tipului întreg NATURAL (cu valori cuprinse între 0 și limita superioară a intervalului de definiție a tipului INTEGER);
- definiția sub-tipului întreg POSITIVE (cu valori cuprinse între 1 și limita superioară a intervalului de definiție a tipului INTEGER);
- definiția tipului STRING ca vector neconstrâns de caractere (de tipul CHARACTER);
- definiția tipului BIT_VECTOR ca vector neconstrâns de biți.

Specificația pachetului STANDARD este dată în continuare:

```

package STANDARD is
-- Tipuri enumerate predefinite
type BOOLEAN is (FALSE, TRUE);
type BIT is ('0', '1');
type CHARACTER is
    (NUL, SOH, STX, ETX, EOT, ENQ, ACK, BEL,
     BS, HT, LF, VT, FF, CR, SO, SI,
     DLE, DC1, DC2, DC3, DC4, NAK, SYN, ETB,
     CAN, EM, SUB, ESC, FSP, GSP, RSP, USP,
     ' ', '!', '"', '#', '$', '%', '&', '\'',
     '(', ')', '*', '+', ',', '-', '.', '/',
     '0', '1', '2', '3', '4', '5', '6', '7',
     '8', '9', ':', ';', '<', '=', '>', '?',
     '@', 'A', 'B', 'C', 'D', 'E', 'F', 'G',
     'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O',
     'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W',
     'X', 'Y', 'Z', '[', '\', ']', '^', '_',
     '`', 'a', 'b', 'c', 'd', 'e', 'f', 'g',
     'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o',
     'p', 'q', 'r', 's', 't', 'u', 'v', 'w',
     'x', 'y', 'z', '{', '|', '}', '~', 'DEL');
type SEVERITY_LEVEL is (NOTE, WARNING, ERROR, FAILURE);
-- Tipuri numerice predefinite
type INTEGER is range -2_147_483_648 to 2_147_483_647;
type REAL is range -16#0.7FFFFFF8#E+32 to 16#0.7FFFFFF8#E+32;
-- Tipul TIME predefinit
type TIME is range -9_223_372_036_854_775_808
    to 9_223_372_036_854_775_807; -- pentru implementarea
    -- pe 64 de biți
    units fs; -- femtosecundă
    ps = 1000 fs; -- picosecundă
    ns = 1000 ps; -- nanosecundă
    us = 1000 ns; -- microsecundă
    ms = 1000 us; -- milisecundă
    sec = 1000 ms; -- secundă
    min = 60 sec; -- minut
    hr = 60 min; -- oră
end units;
-- Funcția care returnează timpul curent al simulării
function NOW return TIME;
-- Sub-tipuri numerice predefinite
subtype NATURAL is INTEGER range 0 to INTEGER'HIGH;
subtype POSITIVE is INTEGER range 1 to INTEGER'HIGH;
-- Tipuri tablou predefinite
type STRING is array (POSITIVE range <>) of CHARACTER;
type BIT_VECTOR is array (NATURAL range <>) of BIT;
end STANDARD;

```

2.3 Pachetul TEXTIO

2.3.1 Prezentare generală

Acest pachet este, din păcate, unul din punctele slabe ale VHDL. Rolul său este de a furniza primitivele de intrare / ieșire ASCII ale VHDL. Așadar, el joacă un rol foarte important și este utilizat atât pentru interacțiunea utilizator-consolă (de exemplu, introducerea de parametri) cât și pentru scrierea și citirea fișierelor (de exemplu, citirea conținutului unui modul de memorie ROM).

Felul în care este scris amintește de intrările și ieșirile din limbajul PASCAL; însă aspectul cel mai grav îl constituie faptul că *specificația pachetului TEXTIO nu respectă regulile generale VHDL*. Orice tentativă de a compila acest pachet este sortită eșecului (de exemplu, în funcția ENDLINE se găsește un parametru de tip acces, ceea ce este interzis în VHDL, după cum am arătat în cadrul lucrării nr. 10 dedicată sub-programelor).

Constructorii de instrumente VHDL au încercat să remedieze această lipsă de omogenitate, realizând fiecare propriul pachet TEXTIO. A rezultat o lipsă evidentă de portabilitate.

În cazul acestui pachet, clauza **use** nu mai este implicită (la fel ca în cazul pachetului STANDARD), deci pachetul TEXTIO va trebui referit în mod explicit dacă se intenționează realizarea de operații de intrare / ieșire.

Alcătuirea acestui pachet este următoarea:

- Definiția tipului LINE. Acest tip va reprezenta linia de fișier manipulată. El este definit ca un acces la tipul STRING, adică la un vector neconstrâns de caractere;
- Definiția tipului TEXT. Este tipul fișierelor de șiruri de caractere care vor fi manipulate de către acest pachet;
- Definiția tipului SIDE. Mai puțin important, acest tip enumerat permite scrierea aliniată la stânga sau la dreapta;
- Definiția tipului WIDTH. Este un tip de date utilizat pentru lungimea șirurilor de caractere; este de fapt o redenumire a tipului NATURAL;
- Definiția fișierelor de text standard. În funcție de implementare, aici se vor găsi corespondențele dintre fișierele INPUT și OUTPUT și respectiv intrarea de la tastatură și ieșirea la consolă a sistemului. Operațiile de intrare / ieșire efectuate asupra

fișierelor INPUT și OUTPUT constituie așa-numitul „dialog consolă”;

- Definiția primitivelor de citire. Primitiva READLINE efectuează citirea unei linii întregi dintr-un fișier dat. Această linie este returnată ca parametru de ieșire al acestei proceduri. Pe această linie vor opera procedurile READ;
- Cele 16 proceduri de citire (READ) primesc ca intrare linia dată, extrag din ea (dacă este posibil) valoarea de tipul cerut și returnează restul liniei, valoarea citită și, în cazul a 8 dintre ele, o variabilă booleană care arată dacă operația a fost efectuată cu succes;
- Cele 8 tipuri de date care pot constitui obiectul unei operații de citire sunt: BIT, BIT_VECTOR, BOOLEAN, CHARACTER, INTEGER, REAL, STRING și TIME;
- Definiția primitivelor de scriere. Procedura WRITELINE efectuează scrierea într-un fișier a unei linii date ca parametru de intrare. Pentru utilizarea acestei primitive, trebuie ca în prealabil această linie să fie construită prin primitive de citire sau prin mai multe apeluri de proceduri WRITE. Există 8 proceduri WRITE, fiecare dintre ele efectuând scrierea într-o linie a unuia dintre cele 8 tipuri menționate mai sus. Această linie este transmisă în mod **inout**; prin urmare, mai multe proceduri WRITE pot completa o aceeași linie înainte de scrierea sa cu ajutorul procedurii WRITELINE;
- Definiția primitivelor de sfârșit de linie și de sfârșit de fișier. O funcție ENDLINE primește ca intrare o linie și returnează valoarea booleană TRUE dacă s-a ajuns la sfârșitul liniei. În anumite implementări, această funcție este transformată în procedură pentru a i se permite să se încadreze în sintaxa și semantica generală VHDL (parametrul LINE al acestei funcții este de tip acces, ceea ce este interzis în VHDL);
- Funcția ENDFILE este definită pe toate tipurile de fișier și deci și pe tipul TEXT. Ea returnează valoarea booleană TRUE dacă s-a ajuns la sfârșitul fișierului.

În VHDL'87 nu existau primitive de creare, de deschidere sau de închidere a fișierelor. Deschiderea fișierului (sau crearea lui, dacă încă nu exista) era efectuată la declararea tipului fișier. Închiderile se efectuau implicit ori de câte ori era necesar: la fiecare ieșire din blocul în care

variabila era vizibilă, fișierul era închis. O dată cu apariția VHDL '93, aceste constrângeri au fost anulate: se presupune că utilizatorul este suficient de matur pentru a-și gestiona singur acest nou tip de comunicare. Remarcăm deci apariția unei noi modalități de comunicare între procese: *prin fișiere*, nu doar prin semnale.

Observație

Rolul acestor constrângeri în VHDL '87 era acela de a evita comunicația între procese prin intermediul fișierelor. Asemenea manipulări ar fi generat un cod VHDL dependent de implementare, deci ne-portabil. În VHDL '87 era imposibil să scriem într-un fișier din interiorul unui proces, să-l închidem și apoi să-l redeschidem din interiorul altui proces.

Începând cu apariția VHDL '93, aceste lucruri devin posibile și prin urmare au fost introduse 2 noi primitive: `FILE_OPEN` și `FILE_CLOSE`, care sunt declarate în mod automat cu tipul `TEXT`. Alte îmbunătățiri aduse de VHDL '93 sunt: trecerea parametrilor procedurilor `READLINE` și `WRITELINE` în mod **inout** și declararea explicită a funcției `ENDFILE` ca funcție impură.

2.3.2 Specificația pachetului *TEXTIO*

```
package TEXTIO is
-- Câteva definiții de tipuri utile
type LINE is access STRING;
type TEXT is file of STRING;
type SIDE is (RIGHT, LEFT);
subtype WIDTH is NATURAL;

-- Definiția fișierelor de text standard („dialog consolă”)
file INPUT: TEXT is in "STD_INPUT";
file OUTPUT: TEXT is out "STD_OUTPUT";

-- Primitive de citire a tipurilor standard
procedure READLINE (variable F: in TEXT; L: out LINE);

procedure READ(L: inout LINE; VALUE: out BIT; GOOD: out BOOLEAN);
procedure READ(L: inout LINE; VALUE: out BIT);

procedure READ(L: inout LINE; VALUE: out BIT_VECTOR; GOOD: out BOOLEAN);
procedure READ(L: inout LINE; VALUE: out BIT_VECTOR);

procedure READ(L: inout LINE; VALUE: out BOOLEAN; GOOD: out BOOLEAN);
procedure READ(L: inout LINE; VALUE: out BOOLEAN);
```

```

procedure READ(L: inout LINE; VALUE: out CHARACTER; GOOD: out BOOLEAN);
procedure READ(L: inout LINE; VALUE: out CHARACTER);

procedure READ(L: inout LINE; VALUE: out INTEGER; GOOD: out BOOLEAN);
procedure READ(L: inout LINE; VALUE: out INTEGER);

procedure READ(L: inout LINE; VALUE: out REAL; GOOD: out BOOLEAN);
procedure READ(L: inout LINE; VALUE: out REAL);

procedure READ(L: inout LINE; VALUE: out STRING; GOOD: out BOOLEAN);
procedure READ(L: inout LINE; VALUE: out STRING);

procedure READ(L: inout LINE; VALUE: out TIME; GOOD: out BOOLEAN);
procedure READ(L: inout LINE; VALUE: out TIME);

-- Primitive de scriere a tipurilor standard
procedure WRITELINE (F: out TEXT; L: in LINE);
procedure WRITE(L: inout LINE; VALUE: in BIT; JUSTIFIED: in SIDE :=
    RIGHT; FIELD: in WIDTH := 0);
procedure WRITE(L: inout LINE; VALUE: in BIT_VECTOR; JUSTIFIED: in
    SIDE := RIGHT; FIELD: in WIDTH := 0);
procedure WRITE(L: inout LINE; VALUE: in BOOLEAN; JUSTIFIED: in
    SIDE := RIGHT; FIELD: in WIDTH := 0);
procedure WRITE(L: inout LINE; VALUE: in CHARACTER; JUSTIFIED: in
    SIDE := RIGHT; FIELD: in WIDTH := 0);
procedure WRITE(L: inout LINE; VALUE: in INTEGER; JUSTIFIED: in
    SIDE := RIGHT; FIELD: in WIDTH := 0);
procedure WRITE(L: inout LINE; VALUE: in REAL; JUSTIFIED: in SIDE :=
    RIGHT; FIELD: in WIDTH := 0; DIGITS: in NATURAL := 0);
procedure WRITE(L: inout LINE; VALUE: in STRING; JUSTIFIED: in SIDE :=
    RIGHT; FIELD: in WIDTH := 0);
procedure WRITE(L: inout LINE; VALUE: in TIME; JUSTIFIED: in SIDE :=
    RIGHT; FIELD: in WIDTH := 0; UNIT: in TIME := NS);

-- Primitive de sfârșit de linie și de sfârșit de fișier
function ENDLIN(L: in LINE) return BOOLEAN;
function ENDFILE(F: in TEXT) return BOOLEAN; --Opțională (nu apare în
--toate implementările)

end TEXTIO;

```

2.3.3 Exemple de utilizare

Prima problemă cu care se confruntă proiectantul este, de regulă, următoarea: linia de cod de mai jos este sistematic respinsă de compilator, deoarece notația "" desemnează atât valori, cât și tipuri BIT_VECTOR și șiruri de caractere.

```
WRITE (linie, "șir de caractere");
```

Această ambiguitate nu poate fi rezolvată decât de către compilator; prin urmare, proiectantul este cel care trebuie să utilizeze expresii calificate, ca în exemplul de mai jos:

```
WRITE (linie, STRING'("șir de caractere"));
```

Iată în continuare un mic exemplu de pachet care utilizează pachetul TEXTIO. Singura procedură din componența sa efectuează achiziția de la consolă a două valori: una de tip TIME și alta de tip BIT. Acestea sunt returnate apelantului, însă numai după ce se trasează achiziția lor prin scrierea într-un fișier. Pentru simplificare, nu se va face nici o verificare la achiziție: presupunem că operatorul va furniza întotdeauna valoarea adecvată la momentul potrivit.

```
package EXEMPLU is
procedure ACHIZIȚIE-TRAS (DATĂ: out TIME; VALOARE: out BIT);
end EXEMPLU;

use STD.TEXTIO.all; --Referire la pachetul TEXTIO

package body EXEMPLU is
file FIȘIER: TEXT is out "TRACE.VHDL"; -- Fișierul de trasare
procedure ACHIZIȚIE-TRAS (DATĂ: out TIME; VALOARE: out BIT) is
--Variabile de lucru
variable LINIE: LINE;
variable DATA: TIME;
variable VALOAREA: BIT;
begin
--Achiziția unei perechi dată / valoare
READLINE (INPUT, LINIE);
READ (LINIE, DATA);
READ (LINIE, VALOAREA);
--Resetare la valoarea null a variabilei de lucru de tip LINE
LINIE := null;
--Scriere în fișierul de trasare
WRITE (LINIE, STRING'("Perechea dată / valoare introdusă: "));
WRITE (LINIE, DATA);
WRITE (LINIE, STRING'(" "));
WRITE (LINIE, VALOAREA);
WRITELINE (FIȘIER, LINIE);
--Actualizarea parametrilor de retur
DATĂ := DATA;
VALOARE := VALOAREA;
end ACHIZIȚIE-TRAS;
end EXEMPLU;
```

La citire, linia este constituită din două elemente: o valoare de tip TIME și o valoare de tip BIT. Utilizatorul va introduce deci, de exemplu:

10 ns 1

și apoi această linie, citită de procedura READLINE, va fi descompusă (prin intermediul celor două proceduri READ) într-o dată (10 ns) și o valoare de bit ('1').

La scriere, linia este alcătuită din patru elemente. Cu ocazia celor patru apeluri ale procedurii WRITE, se scrie succesiv: un șir de caractere ("Perechea dată / valoare introdusă: "), o dată de tip TIME (aici, 10 ns), un șir de caractere care conține două spații (" ") și o valoare de tip BIT (aici, '1'). O dată construită, această linie este scrisă în fișierul FIȘIER cunoscut în sistemul de operare sub numele de TRACE.VHDL, cu ajutorul procedurii WRITELINE.

Observatii

Două apeluri succesive ale procedurii ACHIZIȚIE_TRAS provoacă scrierea celei de-a doua trasări în continuarea celei dintâi, în fișierul FIȘIER. Dacă declarația acestui fișier s-ar fi efectuat în partea declarativă a procedurii (și nu, ca în exemplul de mai sus, în zona declarativă a pachetului), atunci fișierul ar fi fost redeschis la fiecare apel și nu s-ar fi păstrat decât ultima trasare!

Pentru a scrie o procedură de achiziție mai realistă, ar fi preferabil să utilizăm procedurile READ care returnează o valoare booleană de control (ultimul parametru). Testarea acestui parametru ar permite detectarea erorilor (introducerea unei valori de alt tip decât cel așteptat) și tratarea lor (de exemplu, prin declanșarea unei noi achiziții).

2.4 Pachetul STD_LOGIC_1164

2.4.1 Interpretarea tipului logic: semantica pentru simulare

Acest pachet definește logica cu nouă stări; el trebuie să facă parte din biblioteca IEEE. Așadar, toate unitățile de compilare care fac referire la acest pachet trebuie să înceapă cu următoarele clauze **library** și **use**:

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
```

Ansamblul pachetului este bazat pe definiția tipului enumerat numit STD_ULOGIC (sau STD_LOGIC, atunci când este rezolvat). Cele nouă stări au fost deja prezentate în cadrul lucrării nr. 8 referitoare la semnale multi-sursă; ele sunt următoarele:

```
type STD_ULOGIC is (      'U',  --Neinițializat
                           'X',  --Necunoscut tare
                           '0',  --0 tare
                           '1',  --1 tare
                           'Z',  --Înaltă impedanță
                           'W',  --Necunoscut slab
                           'L',  --0 slab
                           'H',  --1 slab
                           '- ', --Fără importanță
                           );
```

Valoarea 'U' fiind prima din tipul enumerat STD_ULOGIC, ea va fi valoarea luată implicit pentru toate obiectele declarate de acest tip ('U' vine de la „Uninitialized”). Semantica sa este simplă: dacă această valoare apare în timpul unei simulări, înseamnă că obiectul respectiv nu a fost inițializat sau cu alte cuvinte nu i s-a asignat nici o valoare.

Semantica valorii 'X' este asemănătoare: ea arată că este imposibil să se definească o valoare '0' sau '1' sau 'Z' (de exemplu ca urmare a unui conflict între mai multe valori asignate unui semnal multi-sursă). Singura informație disponibilă este că obiectul a fost inițializat.

Valorile '0', '1' și 'X' dispun de valori asociate numite "slabe": respectiv, 'L', 'H' și 'W'. Scopul acestei duplicări este acela de a lua în considerare niveluri electrice mai fine decât cele necesare reprezentării lui '0' și '1' logic. De exemplu, o magistrală cu o rezistență „pull-up”¹ va avea, în starea de repaus, valoarea 'H'. Singura semantică oficială este de fapt cea rezumată în tabelul de mai jos (care este de fapt, în același timp, și funcția de rezoluție a semnalelor de acest tip STD_LOGIC, deja prezentată în cadrul lucrării nr. 8 referitoare la domeniul concurent în VHDL).

¹ Se spune despre un semnal care are o rezistență legată la borna de alimentare de potențial pozitiv. În consecință, în absența oricărei comenzi (de exemplu, în cazul unei intrări neconectate), semnalul ia valoarea logică '1'.

U	X	0	1	Z	W	L	H	-		
('U', 'U', 'U', 'U', 'U', 'U', 'U', 'U', 'U'), --	U									
('U', 'X', 'X', 'X', 'X', 'X', 'X', 'X', 'X'), --	X									
('U', 'X', '0', 'X', '0', '0', '0', '0', 'X'), --	0									
('U', 'X', 'X', '1', '1', '1', '1', '1', 'X'), --	1									
('U', 'X', '0', '1', 'Z', 'W', 'L', 'H', 'X'), --	Z									
('U', 'X', '0', '1', 'W', 'W', 'W', 'W', 'X'), --	W									
('U', 'X', '0', '1', 'L', 'W', 'L', 'W', 'X'), --	L									
('U', 'X', '0', '1', 'H', 'W', 'W', 'H', 'X'), --	H									
('U', 'X', 'X', 'X', 'X', 'X', 'X', 'X', 'X'), --	-									

Observații

'X', 'W' și 'U' se numesc *valori meta-logice* și natura lor este diferită de cea a celorlalte semnale. Proiectantul trebuie să manifeste prudență în cazul utilizării lor. De exemplu, rezultatul unei simple comparații poate fi eronat: expresia 'U' < '0' este întotdeauna adevărată (conform semanticii de simulare), datorită definiției tipului enumerat, însă această expresie nu are un sens real.

Valoarea '-' („fără importanță”) nu are nici o semnificație pentru simulare și ea nu trebuie să apară niciodată în urma rezultatelor unei simulări. Utilizarea sa este rezervată sintezei: ea le oferă instrumentelor de sinteză posibilitatea de a codifica această valoare în '0' sau '1', în funcție de optimizările urmărite.

2.4.2 Descrierea pachetului STD_LOGIC_1164

Acest pachet exportă tipul enumerat cu nouă stări, precum și vectorii și sub-tipurile derivate, însoțite de o funcție de rezoluție. Operatorii logici sunt supraîncărcați și se exportă de asemenea și funcții de conversie de tip. Declarația acestui pachet este următoarea:

```

package STD_LOGIC_1164 is

--Declarația tipului enumerat, fundamentul logicii cu nouă stări
type STD_ULOGIC is ( 'U', --Neinițializat
                      'X', --Necunoscut tare
                      '0', --0 tare
                      '1', --1 tare
                      'Z', --Înaltă impedanță
                      'W', --Necunoscut slab
                      'L', --0 slab
                      'H', --1 slab
                      '- ', --Fără importanță
                      );

--Tablou neconstrâns de STD_ULOGIC utilizat în funcția de rezoluție
type STD_ULOGIC_VECTOR is array (NATURAL range <>) of STD_ULOGIC;

-- Funcția de rezoluție: comportamentul acestei funcții nu face parte
-- din specificația pachetului, ea este însă normalizată
function RESOLVED(S: STD_ULOGIC_VECTOR) return STD_ULOGIC;

-- Tipul rezolvat STD_LOGIC este cel care apare cel mai adesea în
-- definițiile modelelor industriale. Din rațiuni de simplitate, se
-- practică o utilizare abuzivă a tipului rezolvat chiar și pentru
-- un semnal uni-sursă.
subtype STD_LOGIC is RESOLVED STD_ULOGIC;

-- Tablou neconstrâns de STD_LOGIC pentru declarații de tablouri
-- de semnale rezolvate
type STD_LOGIC_VECTOR is array (NATURAL range <>) of STD_LOGIC;

--Alte definiții de sub-tipuri
subtype X01 is RESOLVED STD_ULOGIC range 'X' to '1'; --'X','0','1'
subtype X01Z is RESOLVED STD_ULOGIC range 'X' to 'Z'; --'X','0','1','Z'
subtype UX01 is RESOLVED STD_ULOGIC range 'U' to '1'; --'U','X','0','1'
subtype UX01Z is RESOLVED STD_ULOGIC range 'U' to 'Z';--'U','X','0','1','Z'

--Supraîncărcarea operatorilor
function "and" (L: STD_ULOGIC; R: STD_ULOGIC) return UX01;
function "nand" (L: STD_ULOGIC; R: STD_ULOGIC) return UX01;
function "or" (L: STD_ULOGIC; R: STD_ULOGIC) return UX01;
function "nor" (L: STD_ULOGIC; R: STD_ULOGIC) return UX01;
function "xor" (L: STD_ULOGIC; R: STD_ULOGIC) return UX01;
function "not" (L: STD_ULOGIC) return UX01;

--Supraîncărcarea operatorilor pe vectori
function "and" (L, R: STD_ULOGIC_VECTOR) return STD_LOGIC_VECTOR;
function "and" (L, R: STD_ULOGIC_VECTOR) return STD_ULOGIC_VECTOR;
function "nand" (L, R: STD_ULOGIC_VECTOR) return STD_LOGIC_VECTOR;
function "nand" (L, R: STD_ULOGIC_VECTOR) return STD_ULOGIC_VECTOR;
function "or" (L, R: STD_ULOGIC_VECTOR) return STD_LOGIC_VECTOR;
function "or" (L, R: STD_ULOGIC_VECTOR) return STD_ULOGIC_VECTOR;
function "nor" (L, R: STD_ULOGIC_VECTOR) return STD_LOGIC_VECTOR;
function "nor" (L, R: STD_ULOGIC_VECTOR) return STD_ULOGIC_VECTOR;
function "xor" (L, R: STD_ULOGIC_VECTOR) return STD_LOGIC_VECTOR;
function "xor" (L, R: STD_ULOGIC_VECTOR) return STD_ULOGIC_VECTOR;
function "not" (L: STD_ULOGIC_VECTOR) return STD_LOGIC_VECTOR;

```

```

function "not" (L: STD_ULOGIC_VECTOR) return STD_ULOGIC_VECTOR;

--Funcții de conversie
function TO_BIT (S: STD_ULOGIC; XMAP: BIT:='0') return BIT;
function TO_BITVECTOR (S: STD_LOGIC_VECTOR; XMAP: BIT:='0') return
    BIT_VECTOR;
function TO_BITVECTOR (S: STD_ULOGIC_VECTOR; XMAP: BIT:='0') return
    BIT_VECTOR;
function TO_STDULOGIC (B: BIT) return STD_ULOGIC;
function TO_STDLOGICVECTOR (B: BIT_VECTOR) return STD_LOGIC_VECTOR;
function TO_STDLOGICVECTOR (S: STD_ULOGIC_VECTOR) return STD_LOGIC_VECTOR;
function TO_STDULOGICVECTOR (B: BIT_VECTOR) return STD_ULOGIC_VECTOR;
function TO_STDULOGICVECTOR (S: STD_LOGIC_VECTOR) return STD_ULOGIC_VECTOR;

--Transformarea codificării și conversii de tip
function TO_X01 (S: STD_LOGIC_VECTOR) return STD_LOGIC_VECTOR;
function TO_X01 (S: STD_ULOGIC_VECTOR) return STD_ULOGIC_VECTOR;
function TO_X01 (S: STD_ULOGIC) return X01;
function TO_X01 (B: BIT_VECTOR) return STD_LOGIC_VECTOR;
function TO_X01 (B: BIT_VECTOR) return STD_ULOGIC_VECTOR;
function TO_X01 (B: BIT) return X01;
function TO_X01Z (S: STD_LOGIC_VECTOR) return STD_LOGIC_VECTOR;
function TO_X01Z (S: STD_ULOGIC_VECTOR) return STD_ULOGIC_VECTOR;
function TO_X01Z (S: STD_ULOGIC) return X01Z;
function TO_X01Z (B: BIT_VECTOR) return STD_LOGIC_VECTOR;
function TO_X01Z (B: BIT_VECTOR) return STD_ULOGIC_VECTOR;
function TO_X01Z (B: BIT) return X01Z;
function TO_UX01 (S: STD_LOGIC_VECTOR) return STD_LOGIC_VECTOR;
function TO_UX01 (S: STD_ULOGIC_VECTOR) return STD_ULOGIC_VECTOR;
function TO_UX01 (S: STD_ULOGIC) return UX01;
function TO_UX01 (B: BIT_VECTOR) return STD_LOGIC_VECTOR;
function TO_UX01 (B: BIT_VECTOR) return STD_ULOGIC_VECTOR;
function TO_UX01 (B: BIT) return UX01;

--Detectarea fronturilor
function RISING_EDGE (signal S: STD_ULOGIC) return BOOLEAN;
function FALLING_EDGE (signal S: STD_ULOGIC) return BOOLEAN;

--Funcție de testare a obiectelor conținând o valoare necunoscută 'X'
function IS_X (S: STD_ULOGIC_VECTOR) return BOOLEAN;
function IS_X (S: STD_LOGIC_VECTOR) return BOOLEAN;
function IS_X (S: STD_ULOGIC) return BOOLEAN;
end STD_LOGIC_1164;
    
```

2.4.3 Interpretarea tipului logic: semantica pentru sinteză

Scopul grupului de normalizare însărcinat cu definirea acestui pachet era de a defini o semantică precisă pentru fiecare valoare a tipului enumerat STD_ULOGIC și de a propune o interpretare unică și simplă în vederea utilizării sale.

Valorile '0' și '1' trebuie interpretate de către instrumentele de sinteză, respectiv, „conectat la masă” și „conectat la alimentare”. Nu există

semantică definită pentru valorile ‘L’ și ‘H’: instrumentele de sinteză sunt libere să le interpreteze cum doresc. De exemplu, este posibil ca ‘L’ să fie interpretat drept ‘0’ și ca ‘H’ să fie interpretat ca ‘1’.

Observație

Luarea în considerare a valorilor implicite sau a valorilor inițiale constituie o problemă foarte serioasă care apare la sinteză. Unii proiectanți le utilizează în modelele lor ca niște valori care apar ca urmare a unui semnal de *reset*. Cu toate acestea, pentru aceste valori, nu există nici o interpretare standard unanim recunoscută. Proiectanții trebuie să fie conștienți că un model scris astfel *nu este portabil* de la un instrument de sinteză la altul. Dacă funcționalitatea modelului depinde de valorile inițiale, inițializarea acestora trebuie să fie făcută în mod explicit ca urmare a apariției unui eveniment.

Pentru celelalte valori ale tipului STD_ULOGIC, ‘Z’, ‘U’, ‘W’ și ‘X’ (valoarea ‘-’ va fi studiată separat), semnificațiile respective depind de utilizarea acestor valori. Se pot distinge patru cazuri:

1. Valoarea este utilizată ca valoare implicită sau ca valoare inițială. În acest caz nu este definită nici o semantică (a se vedea observația de mai sus);
2. Valoarea este asignată unei variabile sau unui semnal. În acest caz, valorile ‘U’, ‘W’ și ‘X’ vor fi considerate „fără importanță”. Valoarea „fără importanță” (‘-’) poate fi înlocuită la sinteză fie cu un ‘0’ fie cu un ‘1’. Valoarea ‘Z’ va solicita instrumentului de sinteză generarea unui amplificator cu trei stări (*tri-state*). Ieșirea acestui amplificator va constitui ținta instrucțiunii de asignare;
3. Valoarea este utilizată într-o comparație implicită prin intermediul instrucțiunii **case**. Instrucțiunile care apar atunci sunt ignorate (aici se vede foarte clar diferența de semantică dintre sinteză și simulare);
4. Valoarea este utilizată de către un operator de comparare ca expresie a unei condiții în cadrul unei instrucțiuni **if**. Expresiile $A = B$ și $A \neq B$ returnează FALSE, respectiv TRUE dacă unul dintre operanzi utilizează una dintre valorile: ‘U’, ‘W’ sau ‘X’. Comparațiile realizate cu ajutorul operatorilor „>”, „>=”, „<”, „<=” returnează, toate, FALSE.

Valoarea „fără importanță” este foarte utilă și este foarte utilizată în cadrul sintezei. Ea permite două funcționalități:

1. Atunci când valoarea se află în membrul drept al unei instrucțiuni de asignare, este posibilă optimizarea.

```
S <= '-';
V := '-';
ARRAY_S <= "----";
```

Instrumentul de sinteză este liber să înlocuiască fiecare valoare ‘-’ fie cu un ‘1’, fie cu un ‘0’. Scopul acestei flexibilități este de a oferi posibilitatea de a optimiza hardware-ul generat.

2. Când valoarea apare în contextul unei comparații, ea este luată drept valoare generică (*wild card*). Acest lucru poate apărea implicit în cadrul unei instrucțiuni **case** sau explicit în cadrul unei instrucțiuni **if** sau în cadrul unei expresii care întrebunțează un operator de comparare. Valoarea generică permite construirea de filtre. De exemplu, “1101” și “1--1” se potrivesc (match) deoarece comparația este filtrată (comparația nu se realizează efectiv decât asupra primului și ultimului bit). Valorile “1101” și “10-1” nu se potrivesc din cauza celui de-al doilea bit.

Este ușor de intuit cât de importantă este această posibilitate de construcție a filtrelor pentru optimizarea resurselor hardware generate în urma procesului de sinteză. Din păcate, ea introduce o importantă diferență de semantică între sinteză și simulare, astfel încât rezultatele obținute la simularea efectuată înainte și după sinteză riscă să difere considerabil.

În urma scrierii expresiei “1101” = “1--1”, evaluarea sa va returna valoarea FALSE la simulare, dar în urma evaluării aceleiași expresii de către un instrument de sinteză se va obține valoarea TRUE! Firește, astfel se va obține un comportament diferit la simularea pre și post sinteză.

Pentru evitarea unei asemenea situații, se recomandă utilizarea unei funcții specifice numite STD_MATCH, care a fost definită în cadrul unui pachet normalizat IEEE 1076.3 NUMERIC_STD. Principiul acestei funcții constă în aplicarea unei semantici de valoare generică valorii ‘-’. În consecință, comparațiile în care va apărea această valoare vor avea semantici coerente la simulare și la sinteză.

În exemplul de mai jos, rezultatul obținut în urma sintezei va fi același și pentru linia 1 și pentru linia 2, însă numai linia 2 va fi coerentă la simulare atât înainte cât și după sinteză:

```

signal A: STD_ULOGIC_VECTOR (3 downto 0);
...
if A /= "10--" then          -- Linia 1
...
if STD_MATCH(A, "10--") then -- Linia 2
...

```

Se recomandă insistent utilizarea funcției STD_MATCH. În cadrul pachetului normalizat NUMERIC_STD au fost definite mai multe supraîncărcări ale acestei funcții (STD_ULOGIC_VECTOR, STD_ULOGIC, STD_LOGIC, STD_LOGIC_VECTOR, UNSIGNED și SIGNED), iar valoarea returnată este de tip BOOLEAN.

2.5 Pachete aritmetice

Un pachet aritmetic are întotdeauna următoarea structură:

- definiția celor două tipuri numerice SIGNED și UNSIGNED;
- declararea operațiilor aritmetice, a comparațiilor, a deplasărilor (*shift*) și a rotațiilor (*rotate*);
- declararea unui ansamblu de funcții de conversie care permit trecerea de la domeniul vectorilor de biți spre întregi, și invers.

2.5.1 Declarații de tipuri

În cazul pachetului NUMERIC_BIT sunt făcute următoarele declarații:

```

type UNSIGNED is array (NATURAL range <>) of BIT;
type SIGNED   is array (NATURAL range <>) of BIT;

```

În cazul pachetului NUMERIC_STD sunt făcute următoarele declarații:

```

type UNSIGNED is array (NATURAL range <>) of STD_LOGIC;
type SIGNED   is array (NATURAL range <>) of STD_LOGIC;

```

Observații

Cele două tipuri SIGNED și UNSIGNED sunt definite ca tipuri noi și NU ca sub-tipuri ale lui BIT_VECTOR sau STD_LOGIC_VECTOR. Grație acestei precauții, obiectelor de tipul SIGNED sau UNSIGNED nu li se vor putea asigna valori decât cu ajutorul procedurilor sau funcțiilor puse special în acest scop la dispoziția proiectanților. În caz contrar, ar fi fost posibilă amestecarea obiectelor de tip SIGNED sau UNSIGNED, fără a avea un control formal asupra lor.

Se observă, în cazul pachetului NUMERIC_STD, că a fost preferat tipul STD_LOGIC în locul lui STD_ULONG, datorită faptului că primul este mai general, fiind rezolvat, și astfel poate fi aplicat fără constrângeri în orice context. Ideal ar fi fost să se ofere ambele posibilități, dar această abordare ar fi impus definirea de multe alte definiții de tipuri și de funcții asociate, ceea ce ar fi complicat foarte mult contextul general de lucru.

2.5.2 Declarații de operatori

Toți operatorii aritmetici și de comparare definiți pe întregi sunt supraîncărcați pe tipurile SIGNED și UNSIGNED. Astfel, sunt redefiniți operatorii „+”, „-“, „*”, „/”, „rem”, „mod”, „>”, „>=”, „<”, „<=”. Pentru toți acești operatori binari, supraîncărcarea este completă, oferind utilizatorului un maximum de suplețe și de ușurință în exploatare. Spre exemplu, prezentăm supraîncărcarea operatorului binar de adunare:

function „+” (L, R: UNSIGNED)	return UNSIGNED;
function „+” (L, R: SIGNED)	return SIGNED;
function „+” (L: UNSIGNED; R: NATURAL)	return UNSIGNED;
function „+” (L: NATURAL; R: UNSIGNED)	return UNSIGNED;
function „+” (L: INTEGER; R: SIGNED)	return SIGNED;
function „+” (L: SIGNED; R: INTEGER)	return SIGNED;

Dimensiunea rezultatului este întotdeauna specificată în mod foarte precis. Astfel, în cazul adunării (și la fel stau lucrurile și pentru scădere), dimensiunea rezultatului este întotdeauna dimensiunea celui mai mare dintre cei doi vectori primiți ca parametri sau dimensiunea singurului vector primit ca parametru (în cazul în care celălalt operand este de tipul INTEGER sau NATURAL).

În cazul înmulțirii, dimensiunea rezultatului trebuie să fie egală cu suma dimensiunilor celor doi operanzi atunci când aceștia sunt vectori.

Atunci când numai un operand este vector (celălalt fiind un întreg), dimensiunea rezultatului este egală cu dublul dimensiunii operandului vector (se presupune deci că operandul întreg este mai întâi tradus într-un vector de dimensiunea celui alt operand de natură vectorială).

În ceea ce privește împărțirile, dimensiunea rezultatului este egală cu dimensiunea primului operand (deîmpărțitul) atunci când acesta este un vector. În cazul contrar (deîmpărțitul este de tip INTEGER sau NATURAL), dimensiunea rezultatului este luată implicit egală cu dimensiunea împărțitorului (cel de-al doilea operand). În acest ultim caz, conversia deîmpărțitului se face ținând cont de dimensiunea împărțitorului, la nevoie efectuându-se o trunchiere.

Operațiile *modulo* (**mod**) și „*restul împărțirii întregi*” (**rem**) sunt absolut analoage împărțirii în privința calculării și dimensiunii rezultatului (numărul calculat este întotdeauna mai mic decât împărțitorul). Rezultă de aici că dimensiunea rezultatului este egală cu dimensiunea celui de-al doilea operand (împărțitorul) atunci când acesta este un vector. În caz contrar (dacă împărțitorul este de tip INTEGER sau NATURAL), atunci dimensiunea rezultatului este luată implicit drept dimensiunea deîmpărțitului (primul operand). În această situație, conversia împărțitorului se efectuează ținând cont de dimensiunea deîmpărțitului, la nevoie efectuându-se o trunchiere.

Comparațiile sunt efectuate cu operanzi de dimensiuni eventual diferite (dacă operanzii respectivi sunt vectori).

Deplasările și rotațiile sunt definite simultan ca funcții și ca operatori. Motivul este evident: operatorii **sll**, **srl**, **rol** și **ror** nu au apărut decât o dată cu versiunea VHDL'93; deci, pentru păstrarea compatibilității cu vechea versiune VHDL'87, au fost declarate și funcțiile SHIFT_LEFT, SHIFT_RIGHT, ROTATE_LEFT și ROTATE_RIGHT. Dimensiunea rezultatului trebuie să fie identică cu dimensiunea vectorului asupra căruia se efectuează deplasarea sau rotația.

2.5.3 Declarațiile funcțiilor de conversie

Funcțiile de conversie definite în pachetele aritmetice sunt de două feluri:

1. *Funcții care permit trecerea de la o reprezentare vectorială la o altă reprezentare*, neschimbându-se decât dimensiunea rezultatului față de dimensiunea argumentului. Pe scurt, aceste funcții efectuează fie o *extensie a numărului* (dimensiunea rezultatului este mai mare decât dimensiunea argumentului de intrare), fie o *trunchiere* (dimensiunea rezultatului este mai

mică decât dimensiunea argumentului de intrare). Modul de traducere al extensiei diferă, în funcție de argument. Dacă argumentul este de tip SIGNED, atunci bitul de semn (bitul cel mai semnificativ al vectorului argument) este duplicat de atâtea ori cât este necesar pentru a se ajunge la dimensiunea rezultatului dorit. Dacă argumentul este de tip UNSIGNED, atunci se va insera un nou bit de valoare '0' la stânga vectorului argument, de câte ori este necesar, până se ajunge la dimensiunea rezultatului dorit. Există două asemenea funcții, una pentru transformările elementelor de tip SIGNED, cealaltă pentru transformările elementelor de tip UNSIGNED. Specificațiile lor sunt:

```
function RESIZE (ARG: SIGNED; NEW_SIZE: NATURAL) return SIGNED;
function RESIZE (ARG: UNSIGNED; NEW_SIZE: NATURAL) return UNSIGNED;
```

2. *Funcții care permit trecerea de la o reprezentare sub formă de vectori (SIGNED sau UNSIGNED) la o reprezentare sub formă de întregi (INTEGER sau NATURAL) și invers. Funcțiile respective sunt:*

```
function TO_INTEGER(ARG: UNSIGNED) return NATURAL;
function TO_INTEGER(ARG: SIGNED) return INTEGER;
function TO_UNSIGNED(ARG, SIZE: NATURAL) return UNSIGNED;
function TO_SIGNED(ARG: INTEGER; SIZE: NATURAL) return SIGNED;
```

2.5.4 Declarații diverse

Toate funcțiile logice au fost supraîncărcate pe tipurile SIGNED și UNSIGNED.

Există o funcție care permite forțarea la valoarea '0' sau '1' a oricărui element al unui vector de tip SIGNED și UNSIGNED. Această funcție se numește TO_01. Parametrul XMAP, având implicit valoarea '+', permite determinarea valorii finale a elementelor unui vector ale cărui valori nu sunt '0', '1', 'L' sau 'H'.

Mai multe detalii referitoare la pachetele standard și nestandardizate se găsesc în Anexă.

3. Desfășurarea lucrării

3.1 Se va testa funcția NOW din cadrul pachetului STANDARD.

- 3.2 Se va testa funcția de rezoluție a tipului STD_LOGIC prin crearea unui semnal multi-sursă și luarea în considerație a tuturor cazurilor.
- 3.3 Se vor utiliza funcțiile de conversie predefinite în cadrul pachetelor aritmetice pentru implementarea unui numărător bidirecțional pe 8 biți.